

Hyper parameter Tuning

- **Hyperparameter tuning** is the process of **optimizing** the **hyperparameters** of a machine learning model to improve its **performance**.
- Hyperparameters are set before the **training process** and **influence** how the **model learns from the data**.
- The goal of hyperparameter tuning is to find the **best combination of hyperparameters** that minimize the **model's error** on **validation data** (e.g., **reduce loss or increase accuracy**) and improve its **generalization ability on unseen data**.

Tuning

1. Grid Search:

This method systematically tries **every combination of hyperparameter** values within a defined search space.

2. Random Search:

Instead of trying every combination, this method **samples hyperparameters randomly** from the search space.

Loss Function

- A **loss function** measures how well a model's predictions match the actual target values.
- The goal of **training a machine learning model** is to **minimize this loss function**, which in turn improves the model's performance.
- During training, **optimization algorithms** like **gradient descent** adjust the **model parameters** to **minimize the loss function**.

Types of Loss Function

1. **Mean Squared Error (MSE)** — for Regression
2. **Cross-Entropy Loss (Log Loss)** — for Classification
3. **Hinge Loss** — for Support Vector Machines (SVM)

Regularization

- **Regularization** in machine learning is a technique used to **prevent overfitting**, which occurs when a **model learns** not only the **underlying patterns in the training** data but also the **noise or irrelevant details**.
- Overfitting can lead to **poor generalization** performance on **unseen data**.
- Regularization **adds a penalty** to the **loss function** as in **L1 or L2** regularization, the model is encouraged to keep the **weights smaller**.
- This **regularization** helps the model focus on the **most important patterns** in the **data**, improving its ability to make **accurate predictions** on unseen data and ensuring better **generalization**.

There are several types of regularization

L1 Regularization(Lasso):

- During training, **L1 regularization** encourages the model to **shrink** the **less important feature weights(Coefficient)** to **zero**.
- As a result, only the **most important features** retain **non-zero weights**.
- This property makes **L1 regularization** useful for **feature selection**.

.....

L2 regularization(RIDGE):

Instead of setting **weights** to **exactly zero** as in L1 regularization, L2 regularization **shrinks all weights** by a proportional amount, ensuring that **they remain small but non-zero.**

Elastic Net:

Combines both **L1** and **L2** regularization. It strikes a **balance** between **feature selection** and **shrinking the coefficients.**

None: No regularization

Advanced Optimization Algorithms (Solvers)

Besides **basic gradient descent**, logistic regression can also use **more sophisticated optimization algorithms**.

logistic regression uses **different solvers** depending on the size of the **dataset, regularization type, and computational efficiency**:

1. **liblinear**: For small datasets and L1/L2 regularization.
2. **lbfgs**: For large datasets and L2 regularization (recommended for most cases).
3. **saga**: For large datasets with both L1 and L2 regularization, and for sparse data.
4. **newton-cg**: For larger datasets with L2 regularization.

Regularization Strength **C**

C hyperparameter controls the strength of regularization.

When **C is small**:

- The model focuses more on regularization (**keeping the weights small**).
- It can **sacrifice some accuracy** on the **training data** in exchange for **better generalization on unseen data**.
- This can help in **avoiding overfitting**.

■ ■ ■ ■ ■ ■ ■

When **C is large:**

The model gives **less importance** to **regularization** and tries to **fit** the **training data** as **closely** as possible.

This can lead to **overfitting**, where the model **learns noise** and **specific patterns** in **the training data** that don't **generalize** to new data.

- • •
- The values in **c_values=[100, 10, 1.0, 0.1, 0.01]** represent different **C hyperparameter values** used in regularization for a model like Logistic Regression.
 - **C = 100:** Weak regularization (strong focus on fitting the training data).
 - **C = 10:** Slightly stronger regularization than 100, but still focuses more on fitting the data closely.
 - **C = 1.0:** This is a balanced value, typically used as a default. It applies moderate regularization.
 - **C = 0.1:** Stronger regularization (more emphasis on keeping weights smaller).
 - **C = 0.01:** Very strong regularization (much more focus on simplifying the model by shrinking the weights).